Observability and the Course Sessions Wrap

Satyadhyan Chickerur, Ph.D., MBA(Edu.Mgmt), SMIEEE
Director, Centre for Artificial Intelligence Research,
Prof., School of Computer Science & Engineering,
KLE Technological University,
BVB Campus, Vidyanagar,
Hubballi - 580031, Karnataka, India

The Big Picture — Why This Session Exists

A pipeline can run perfectly, report success, and still silently produce garbage. Observability is how you catch that before a customer does.

THINK OF IT LIKE THIS: A car's dashboard doesn't just have an 'engine on' light — it has a speedometer, fuel gauge, and warning lights for specific problems. 'The job ran' is the engine light. Observability is the whole dashboard: it tells you HOW WELL it ran, not just whether it started.

1 Data pipelines fail silently

The job 'succeeds' but the output is empty, wrong, or stale.

2 Three complementary signals

Logs, metrics, and traces each answer a different question.

3 Service Level Objectives (SLOs) make 'healthy' measurable

An explicit promise, not a vague feeling.

Key Terms Before We Start

Log — A discrete, timestamped record of something that happened.

Metric — A numeric measurement tracked over time, like rows processed per minute.

Trace — A record of where time was spent across the steps of one request or run.

SLO — An explicit, measurable promise about how well a system should perform.

Error budget — The amount of 'bad' (downtime, errors) you're allowed before you must slow down and fix things.

Why Observability Matters for Data Pipelines

Empty outputs

Job ran fine. Wrote 0 rows. Filter logic bug.

Stale data

Yesterday's run failed silently; today uses 2-day-old data.

Schema drift

Upstream added a column; your job ignored it.

Volume drops

1B rows yesterday, 100M today. Why?

Distribution shift

Same row count, different values — model degrades.

Quiet skew

Job succeeds but takes 10x longer, quietly.

The Three Pillars — Logs, Metrics, Traces

LOGS

USED FOR

What happened, when, where.

EXAMPLE

```
ERROR partition 47 OOM
```

METRICS

USED FOR

Trends, alerts, SLOs.

EXAMPLE

```
rows_processed_total = 12.4B
```

TRACES

USED FOR

Where time was spent.

EXAMPLE

```
shuffle_stage took 47s
```

Together they answer 'what, how much, where.' One alone answers none. A real investigation uses all three, in that order.

Instrumenting a Pipeline — the Basics

```
rows_in = Counter('etl_rows_in_total', 'rows read', ['stage'])
stage_seconds = Histogram('etl_stage_seconds', 'duration',
['stage'])

def run_stage(name, fn, df):
    with tracer.start_as_current_span(name):
        n_in = df.count(); out = fn(df); n_out = out.count()
        logging.info('stage_done', extra={'stage': name,
'rows_in': n_in,
            'rows_out': n_out, 'drop_rate': 1 -
n_out/max(n_in,1)})
        return out
```

KEY DETAIL: drop_rate = (rows_in - rows_out)/rows_in. That field is what catches 'the filter accidentally dropped 50% of users' silently. Alert on drop_rate > 0.1 unexpectedly.

SLOs for Data Pipelines — Four Dimensions

Freshness

Max age of latest data. Ex: feature table under 30 min old.

Completeness

Fraction of expected rows present. Ex: >=99.5% of daily events.

Correctness

Fraction of rows passing schema/value checks.

Latency (p99)

Time from event to availability downstream.

RULE: define SLOs in terms downstream consumers actually feel. Not 'job ran' — 'data was usable.'

Bottleneck Analysis — Reading the Pillars Together

A worked investigation: a nightly ETL that normally takes 40 minutes takes 3 hours tonight.

1. METRICS first: the shuffle stage jumped from 4 min to 2h40 — that's the outlier
2. LOGS next: repeated WARN entries about spill-to-disk on one executor
3. TRACES to confirm: one executor's span is 2h35 while its siblings finish in minutes
4. Cross-reference rows_in/rows_out: that executor's partition has 40x the median row count
5. DIAGNOSIS: data skew from a new bot account in last night's data
6. FIX: apply the salted-groupby pattern, or verify AQE skew handling is enabled

Common Mistakes & How to Spot Them

Mistake	How it shows up	Fix
Only checking 'did the job succeed'	Silent data quality issues go unnoticed for weeks	Track drop_rate, row counts, and freshness explicitly
No SLOs defined anywhere	Nobody can say whether a pipeline is 'healthy'	Define freshness/completeness/ correctness/latency SLOs
Using only one of logs/metrics/traces	Investigations take hours instead of minutes	Instrument all three, use them together

What You Can Now Do

1. Stand up a GPU-enabled containerised dev environment, reproducible across machines
2. Deploy a workload onto Kubernetes with proper probes, resources, and GPU allocation
3. Profile and optimise PyTorch training end to end
4. Train at scale with DDP/FSDP — multi-node, NCCL, debugging rendezvous
5. Ship a model with proper packaging, versioning, and CI/CD
6. Build a distributed data pipeline — choose the right tool, handle skew
7. Reason about cost, scaling, and observability — engineering judgment, not just code